

WAV в твоём проекте

PDF-версия отчёта: [скачать](#).

Что это за WAV на самом деле

В твоём коде WAV — это не «сам звук», а контейнер RIFF/WAVE, внутри которого лежит секция `fmt` с описанием формата и секция `data` с самими аудиоданными. В терминах спецификации RIFF файл состоит из chunk-ов, а для WAVE обязательны как минимум `fmt` и `wave-data`; при этом `fmt` должен идти раньше `data`. То есть на низком уровне твой файл — это «паспорт формата + линейная последовательность PCM-отсчётов». ¹

Ещё один важный слой: выбранный тобой формат — это формат хранения и передачи внутри проекта, а не обязательно финальное представление, которое доходит до модели. Документация LiteRT-LM для Android показывает, что сообщения могут содержать `AudioBytes` и `AudioFile`, а high-level `Conversation` API выполняет preprocessing multimodal data; в C++-документации прямо сказано, что model data processor занимается преобразованием generic message в входные данные модели и включает audio preprocessors для multimodal-моделей. Отдельно документация Gemma по audio input рекомендует приводить аудио к одному каналу, 16 кГц и `float32` в диапазоне `[-1, 1]`, а максимальная длина клипа — 30 секунд. Из этого следует простая инженерная картина: твой `.wav mono 16 kHz int16` — хороший промежуточный артефакт для pipeline, но downstream-обработчик вполне может декодировать WAV и переводить `int16 -> float32` уже перед инференсом. ²

Практически это означает следующее: формулировка из комментария

`WAV / mono / 16 kHz / PCM 16-bit little-endian / max 30 seconds` — это корректная спецификация именно твоего проектного контракта. Но её нельзя путать ни с «определением WAV вообще», ни с «единственным возможным форматом для LiteRT-LM». WAV как контейнер допускает и другие chunk-ы, и другие кодеки, а модельная часть может декодировать и нормализовать сигнал дальше по своим правилам. ³

Как звук становится цифрой

На физическом уровне звук — это непрерывное изменение давления воздуха. Если говорить языком тракта, микрофон превращает эту аналоговую волну в аналоговый электрический сигнал, а АЦП затем преобразует этот сигнал в цифровые слова на частоте дискретизации `Fs`; документация TI по ADC прямо описывает базовую схему как analog signal на входе, который преобразуется в digital words с частотой `Fs` от ADC clock. Это и есть момент, где непрерывная волна становится последовательностью чисел. ⁴

По классической формулировке теоремы дискретизации Шеннона, если сигнал не содержит частот выше `W`, то он полностью определяется отсчётами, взятыми через интервалы `1/(2W)`. В прикладной форме для ADC это обычно записывают как требование: чтобы корректно представить входной сигнал, частота дискретизации должна быть как минимум вдвое выше максимальной значимой частоты сигнала, то есть граница Найквиста находится на `Fs/2`. Для твоих `16 000 Hz` это означает шаг между отсчётами `62.5 μs` и Nyquist-частоту около `8 kHz`.

Для голосового pipeline это не случайный выбор: Google для Gemma отдельно рекомендует single-channel audio с частотой `16 kHz`, так что твой формат не просто «маленький», а ещё и стыкуется с типовым model-side preprocessing. ⁵

Как непрерывный звук превращается в отсчёты и уровни

На картинке выше видна суть процесса: синяя линия — непрерывная волна, оранжевые точки — моменты съёма отсчётов, зелёная «лестница» — уже квантованные уровни. Реальная 16-битная квантовка имеет 65 536 уровней, поэтому визуально она почти гладкая; на схеме уровни специально укрупнены, чтобы показать сам принцип. То, что ты потом кладёшь в WAV, — это не функция `x(t)` целиком, а её приближённая сетка значений по времени. ⁶

Отсюда возникает важная низкоуровневая проблема: если на вход АЦП попадают высокочастотные компоненты вне полезной полосы, они могут «сложиться назад» в рабочий диапазон как aliasing. TI прямо пишет, что analog anti-aliasing filters attenuate out-of-band high-frequency components prior to ADCs и предотвращают folding back в интересующую полосу; в другом tutorial по ADC TI отдельно отмечает, что при `Fin > Fs/2` происходит undersampling, и на выходе получается aliased frequency. Поэтому «звуковой контур» в цифре строится честно только тогда, когда перед дискретизацией тракт ограничивает лишние частоты. ⁷

По глубине квантования у тебя 16 бит. Для идеального `N`-битного АЦП теоретический SNR по квантованию равен `6.02N + 1.76 dB`; для `N = 16` это `98.08 dB`. Важно слово «идеального»: сама статья Analog Devices подчёркивает, что эта формула описывает theoretical performance of a perfect N-bit ADC. Значит, 16 бит в твоём WAV — это хороший контейнерный стандарт и разумный нижний уровень совместимости, но не обещание, что телефонный микрофон, аналоговый фронтенд и системная обработка дадут тебе реальные 98 дБ полезной динамики. Это уже инференс из того, что формула описывает идеальный предел, а не весь смартфонный тракт. ⁸

Что именно лежит в PCM 16-bit little-endian

Теперь к самому мясу. В PCM WAVE у тебя хранятся не «семантические голосовые признаки» и не «частоты», а просто последовательные амплитудные отсчёты. Оригинальная IBM/Microsoft-спецификация WAVE говорит, что в single-channel WAVE samples are stored consecutively, а для stereo samples are interleaved. Она же фиксирует, что для sample size от 9 бит и выше данные хранятся как signed integer, и что least significant byte is stored first. Это ровно твой случай: mono, 16-bit, signed, little-endian. ⁹

Отсюда следует точный смысл твоих констант:

```
const val SAMPLE_RATE_HZ = 16_000
const val CHANNEL_COUNT = 1
const val BITS_PER_SAMPLE = 16
const val BYTES_PER_SAMPLE = BITS_PER_SAMPLE / 8
```

Один аудиофрейм у тебя содержит один сэмпл одного канала, то есть `2` байта. Один сэмпл принимает значения примерно от `-32768` до `32767`; IBM/Microsoft-спецификация для 16-bit PCM прямо приводит midpoint `0`, max `32767 (0x7FFF)` и min `-32768 (-0x8000)`. Поэтому

секция `data` в твоём WAV — это просто цепочка 16-битных чисел, идущих друг за другом без сжатия. ¹⁰

Как выглядят 16-битные little-endian сэмплы

Именно поэтому твой `calculateVoiceLevel()` собирает сэмпл так:

```
val lowByte = buffer[index].toInt() and 0xFF
val highByte = buffer[index + 1].toInt()
val sample = ((highByte shl 8) or lowByte).toShort().toInt()
```

Это очень правильный низкоуровневый код. `lowByte` маскируется через `0xFF`, чтобы убрать знаковое расширение `Byte`; `highByte` остаётся знаковым, потому что именно старший байт несёт знак в двухкомплементарном представлении; затем два байта собираются в 16-битное слово и приводятся к `Short`, чтобы восстановить signed PCM-значение. Например, байты `18 FC` = `0xFC18` = `-1000`, а `E8 03` = `0x03E8` = `+1000`. Общая семантика такого packing-a полностью соответствует WAVE-спецификации: младший байт идёт первым, а 16-битный PCM — знаковый. ¹¹

Дальше ты нормализуешь сэмпл делением на `32768.0`:

```
val normalizedSample = sample / PCM_16_MAX_ABS_VALUE
```

Это тоже хороший инженерный ход. Документация Gemma рекомендует переводить `int16` в `float32` и делить на `32768.0`, чтобы получить диапазон `[-1, 1]`. У этого выбора есть лёгкая асимметрия: `-32768` становится ровно `-1.0`, а `+32767` — примерно `+0.99997`. Но это не дефект, а стандартная практика нормализации signed PCM-контейнера в float-domain. ¹²

Отдельно про RMS-метр. В твоей функции он считается как корень из среднего квадрата окна сэмплов, после чего ты убираешь небольшой noise floor и усиливаешь показания для UI. Это значит, что `voiceLevels` в `KsenaxRecordedVoiceMessage` — это не свойства WAV-файла, не «реальная громкость речи» и не модельные признаки, а отдельная служебная проекция того же PCM-потока, рассчитанная поверх байтового окна. Важно это не путать: WAV хранит исходные отсчёты; `voiceLevels` — это derived telemetry для интерфейса.

Почему твой заголовок ровно 44 байта

Вот главный момент, который почти всегда понимают поверхностно. Фраза «WAV header = 44 bytes» верна не вообще, а только для канонического минимального PCM WAVE. У тебя именно такой случай: `RIFF` chunk, внутри него `WAVE`, затем `fmt` с 16 байтами PCM-описания и `data` chunk header на 8 байт. В старой IBM/Microsoft-спецификации WAVE записан как `RIFF('WAVE' <fmt-ck> ... <wave-data>)`, причём `fmt` должен идти до `wave-data`, оба обязательны, а неизвестные chunk-ы нужно игнорировать. Плюс RIFF в целом допускает и другие chunk-ы, а Microsoft отдельно описывает `fact`, `cue`, `LIST` и прочие расширения. Поэтому 44 байта — это свойство твоего минимального PCM-файла, а не универсальная константа для всех WAV вообще. ¹³

Канонический PCM WAV-заголовок на 44 байта

Семантика каждого поля в твоём `createWavHeader()` совпадает с `WAVEFORMATEX`. Microsoft определяет `nChannels` как число каналов, `nSamplesPerSec` как частоту дискретизации, `nAvgBytesPerSec` как среднюю скорость передачи байт в секунду, `nBlockAlign` как minimum atomic unit of data, а `wBitsPerSample` как bits per sample. Для PCM `nAvgBytesPerSec` должно быть `nSamplesPerSec * nBlockAlign`, а `nBlockAlign` — $(nChannels * wBitsPerSample) / 8$. Старый WAVE-документ говорит то же самое: для PCM `wAvgBytesPerSec` и `wBlockAlign` вычисляются именно из числа каналов, sample size и sample rate. ¹⁴

Твои значения для текущего формата получаются такие:

- `byteRate = 16000 * 1 * 2 = 32000 B/s`
- `blockAlign = 1 * 2 = 2 B`
- `fmt size = 16` для обычного PCM
- `audioFormat = 1`, то есть PCM
- `WAV_STANDARD_HEADER_SIZE = 44`

Это не «магические числа», а прямые следствия структуры RIFF/WAVE и параметров PCM. Поле `totalDataLength = pcmDataSize + 44 - 8` тоже правильно: RIFF size хранит размер данных файла без первых четырёх байт `RIFF` и без самого поля размера. Microsoft Learn формулирует это буквально так: `fileSize` includes the size of `fileType` plus the data that follows, but does not include the size of `"RIFF"` or the size of `fileSize`. ¹⁵

Отсюда можно руками проверить твой заголовок на примере одной секунды тишины:

```
00: 52 49 46 46  -> "RIFF"
04: 24 7D 00 00  -> RIFF size = 32036
08: 57 41 56 45  -> "WAVE"
12: 66 6D 74 20  -> "fmt "
16: 10 00 00 00  -> fmt size = 16
20: 01 00 01 00  -> PCM, mono
24: 80 3E 00 00  -> 16000 Hz
28: 00 7D 00 00  -> 32000 byte/s
32: 02 00 10 00  -> blockAlign=2, bits=16
36: 64 61 74 61  -> "data"
40: 00 7D 00 00  -> data size = 32000
```

Ещё одна низкоуровневая деталь, про которую часто забывают: RIFF пишет chunk data с выравниванием до word boundary, а `chunkSize` не включает pad byte. В твоём конкретном формате это почти не замечается, потому что моно 16-bit PCM и так всегда даёт чётный объём payload-а: `data size = sampleCount * 2`, то есть паддинг обычно не нужен. Но как только формат станет нечётным по размеру данных или появятся дополнительные chunk-ы, это уже важно. ¹⁶

Как код реализует весь тракт на Android

Твой `AudioRecord.Builder()` задаёт формат явно:

```
.setAudioSource(MediaRecorder.AudioSource.MIC)
.setAudioFormat(
    AudioFormat.Builder()
        .setSampleRate(SAMPLE_RATE_HZ)
        .setEncoding(AUDIO_ENCODING)
        .setChannelMask(CHANNEL_CONFIG)
        .build()
)
```

Это правильно архитектурно. Android пишет, что если audio format не задан или incomplete, то по умолчанию будут `CHANNEL_IN_MONO` и `ENCODING_PCM_16BIT`, а sample rate будет зависеть от реально выбранного девайса. Явное задание sample rate / channel mask / encoding делает твой контракт жёстче и уменьшает «магическое поведение» аудиостека. ¹⁷

Дальше ты берёшь `getMinBufferSize()` и потом ставишь внутренний буфер рекордера как минимум `16000` байт. Это хорошо согласуется с документацией `AudioRecord.Builder.setBufferSizeInBytes()`: Android определяет этот размер как total size of the buffer where audio data is written during recording и отдельно подчёркивает, что новые данные можно читать smaller chunks than this size. В твоём коде внутренний headroom получается как минимум полсекунды ($16000 / 32000 \text{ B/s} = 0.5 \text{ s}$), а read-buffer `2048` байт даёт примерно `64 ms` аудио на chunk. Для voice meter-a и отмены по push-to-talk это очень разумная разменная монета между latency и безопасностью к scheduling jitter. ¹⁸

Здесь же важна идея фрейма. `AudioFormat` пишет, что для PCM frame size in bytes равен sample size multiplied by channel count, а `AudioRecord` обрабатывает данные кратно frame size. В твоём формате frame size = `2` байта, и `2048`, и `maxBytes`, и общий payload всегда кратны 2. Это убирает класс проблем с обрезанным half-sample на границе чтения. ¹⁹

Семантика самого чтения у тебя тоже аккуратная. `AudioRecord.read(...)` по документации возвращает либо `0` / positive count, либо error codes; количество байт не превышает requested size и обрезается до целого числа frame-ов. Поэтому проверка `if (readBytes > 0)` и запись только положительного объёма — корректный защитный паттерн. Ты не предполагаешь, что каждый `read()` обязан вернуть полностью запрошенный кусок, и это правильно для реального аудиостека. ²⁰

Но вот тут есть инженерный нюанс, который я бы не пропускал. Android в документации overload-а `read(byte[], ..., readMode)` говорит: для byte array формат should be `ENCODING_PCM_8BIT`; `ENCODING_PCM_16BIT` допускается, но это deprecated. Для `short[]` документация, наоборот, говорит, что constructor format should be `ENCODING_PCM_16BIT`, а для direct `ByteBuffer` — что представление данных зависит от формата `AudioRecord` и будет native endian. То есть твой текущий код обычно работает, но если хочешь сделать реализацию более «по документации» и чуть чище по намерению, то для PCM16 есть два более каноничных пути: `read(short[])` или `read(direct ByteBuffer)`. На мой взгляд, это не баг, а хороший next refinement. ²⁰

Есть и ещё один неочевидный слой: аудиоисточник. `MIC` — это просто microphone audio source. `UNPROCESSED` описан как microphone audio source tuned for unprocessed raw sound if available, иначе ведёт себя как default. `VOICE_RECOGNITION` — источник, tuned for voice recognition. `VOICE_COMMUNICATION` — источник для VoIP и может использовать echo cancellation или automatic gain control, если доступно. Поэтому если твоя конечная задача — не красивый пользовательский meter, а максимально стабильный анализ речевого сигнала для модели, я бы не верил, что `MIC` автоматически оптимален на всех девайсах. Я бы A/B-тестировал как минимум `MIC`, `VOICE_RECOGNITION` и `UNPROCESSED`. ²¹

Где у конструкции границы и что я бы усилил

Первая слабая предпосылка — считать, что «WAV = всегда 44 байта хедера». Нет. Это верно только для минимального PCM-кейса с `RIFF + fmt + data`. Microsoft и старый WAVE-документ прямо допускают дополнительные chunk-ы, а `fact` обязателен для compressed audio formats и для `wav1`-варианта хранения. Более того, Microsoft отдельно пишет, что `WAVEFORMATEX` покрывает лишь subset форматов, а для многоканальных или более сложных кейсов используется `WAVEFORMATEXTENSIBLE`. Если когда-нибудь захочешь метаданные, маркеры, multichannel или расширенный PCM, твой fixed-44-path уже не будет универсальным. ²²

Вторая слабая предпосылка — мыслить так, будто «модель ест именно тот WAV, который ты записал». На деле LiteRT-LM high-level stack абстрагирует мультимодальный input через `Content / Message` и document-ирует preprocessing multimodal data в Conversation API; Gemma отдельно рекомендует после декодирования переходить к mono 16 kHz `float32 [-1, 1]`. То есть у тебя полезно разделить в голове два контракта: **контракт захвата** и **контракт модели**. Первый у тебя уже хороший: простой PCM16 WAV. Второй я бы сделал явно отдельной функцией-конвертером, даже если сейчас библиотека часть работы делает внутри. Тогда ты сможешь независимо контролировать ресемплинг, downmix, нормализацию и trimming. ²³

Третья слабая предпосылка — воспринимать `voicelevels` как характеристику файла. Нет, это характеристика твоего runtime-окна измерения поверх файла. WAV сам по себе хранит линейную последовательность отсчётов; RMS-meter — это уже интерпретация этих отсчётов. Это не проблема, а просто правильная граница ответственности: контейнер хранит сырьё, UI живёт на derived metrics.

Если сжать всё в одну рабочую модель, то она такая: **микрофон даёт непрерывный сигнал, АЦП снимает его сеткой отсчётов, квантование зажимает каждый отсчёт в signed int16, WAV добавляет RIFF-паспорт, а модельный pipeline потом декодирует и переупаковывает эти числа в тот численный формат, который нужен самой модели**. Именно в этом смысле твой код уже довольно чистый: он не делает «магии аудио», он честно строит предсказуемый байтовый артефакт поверх понятной цифровой сетки. ²⁴

¹ ³ ¹⁵ ¹⁶ ²⁴ <https://learn.microsoft.com/en-us/windows/win32/xaudio2/resource-interchange-file-format--riff->

<https://learn.microsoft.com/en-us/windows/win32/xaudio2/resource-interchange-file-format--riff->

² <https://ai.google.dev/edge/litert-lm/android>

<https://ai.google.dev/edge/litert-lm/android>

⁴ <https://www.ti.com/lit/pdf/slaa510>

<https://www.ti.com/lit/pdf/slaa510>

- 5 6 <https://webusers.imj-prg.fr/~antoine.chambert-loir/enseignement/2020-21/shannon/shannon1949.pdf>
<https://webusers.imj-prg.fr/~antoine.chambert-loir/enseignement/2020-21/shannon/shannon1949.pdf>
- 7 <https://www.ti.com/lit/pdf/sbaa377>
<https://www.ti.com/lit/pdf/sbaa377>
- 8 <https://www.analog.com/media/en/training-seminars/tutorials/MT-001.pdf>
<https://www.analog.com/media/en/training-seminars/tutorials/MT-001.pdf>
- 9 10 11 13 22 <https://www.mmsp.ece.mcgill.ca/Documents/AudioFormats/WAVE/Docs/riffmci.pdf>
<https://www.mmsp.ece.mcgill.ca/Documents/AudioFormats/WAVE/Docs/riffmci.pdf>
- 12 <https://ai.google.dev/gemma/docs/capabilities/audio>
<https://ai.google.dev/gemma/docs/capabilities/audio>
- 14 <https://learn.microsoft.com/en-us/windows/win32/api/mmreg/ns-mmreg-waveformatex>
<https://learn.microsoft.com/en-us/windows/win32/api/mmreg/ns-mmreg-waveformatex>
- 17 18 <https://developer.android.com/reference/kotlin/android/media/AudioRecord.Builder>
<https://developer.android.com/reference/kotlin/android/media/AudioRecord.Builder>
- 19 **AudioFormat | API reference | Android Developers**
<https://developer.android.com/reference/android/media/AudioFormat>
- 20 <https://developer.android.com/reference/android/media/AudioRecord>
<https://developer.android.com/reference/android/media/AudioRecord>
- 21 <https://developer.android.com/reference/android/media/MediaRecorder.AudioSource>
<https://developer.android.com/reference/android/media/MediaRecorder.AudioSource>
- 23 <https://github.com/google-ai-edge/LiteRT-LM/blob/main/docs/api/cpp/conversation.md>
<https://github.com/google-ai-edge/LiteRT-LM/blob/main/docs/api/cpp/conversation.md>